

----- MongoDB Queries Concepts Covered -----

[1. DATABASE COMMANDS](#)

[2. CREATE / DELETE OPERATIONS](#)

[3. INSERT / UPDATE COMMANDS](#)

[4. FIND / DISPLAY DATA](#)

[5. SPECIAL CHARACTERS &
RESERVED KEYWORDS](#)

[6. ORDERED AND UNORDERED
INSERT](#)

[7. FINDING DOCUMENTS](#)

[8. COMPARISON OPERATORS](#)

[9. CURSOR METHODS](#)

[10. LOGICAL OPERATORS](#)

[11. COMPLEX EXPRESSIONS \(\\$expr\)](#)

[12. ELEMENT OPERATORS](#)

[13. PROJECTION](#)

[14. EMBEDDED DOCUMENTS](#)

[15. \\$ALL & \\$ELEMATCH](#)

[16. UPDATE OPERATIONS](#)

[17. REMOVE AND RENAME FIELDS](#)

[18. UPDATING ARRAYS AND
EMBEDDED DOCUMENTS](#)

[19. DELETE OPERATIONS](#)

[20. INDEXES](#)

[21. DROP INDEX](#)

[22. AGGREGATION FRAMEWORK](#)

[23. \\$LOOKUP \(JOIN IN
AGGREGATION\)](#)

[24. \\$PROJECT STAGE](#)

[25. \\$GROUP WITH ARRAYS](#)

[26. \\$UNWIND STAGE](#)

[27. \\$FILTER OPERATOR](#)

[28. \\$FACET \(MULTIPLE PIPELINES IN
AGGREGATION\)](#)

[29. TRANSACTIONS \(ACID
OPERATIONS\)](#)

[30. TTL INDEX \(AUTOMATIC DATA
EXPIRY\)](#)

----- DATABASE COMMANDS -----

----- Create / Delete -----

Show all available databases

show dbs

show databases

Create (or switch to) a database named "students"

use students

Check if the new database appears after creating a collection

(Database visible only after adding a collection/document)

show databases

Create a new collection named "data" inside the selected database

db.createCollection('data')

Show all collections in the current database

show collections

show dbs

----- DELETE / DROP COMMANDS -----

`deleteOne()` → deletes only first matching document

`deleteMany()` → deletes all matching documents

Delete or drop a specific collection named "data"

`db.data.drop()`

`show collection`

`show dbs`

Delete or drop the entire current database

`db.dropDatabase()`

`show dbs`

----- INSERT / UPDATE COMMANDS -----

insertMany() returns:

- acknowledged
- insertedCount
- insertedIds

Switch to the "student" database (if not already in it)

use student

Create a collection named "data" in the student database

db.createCollection('data')

Insert a single document into the "data" collection

db.data.insertOne({ 'name' : 'anant', age : 21 })

Insert multiple documents at once into the "data" collection

```
db.data.insertMany([
  { 'name' : 'anant', age : 21 },
  { 'name' : 'ramesh', age : 22 },
  { 'name' : 'mahesh', age : 25 }
])
```

----- FIND / DISPLAY DATA -----

`find()` → returns a cursor (multiple documents)

`findOne()` → returns a single document

Show all data/documents in the collection in array format

`db.data.find()`

----- SPECIAL CHARACTERS / RESERVED KEYWORDS -----

Example of invalid field name due to space (this will cause a syntax error)

When a key/field contains space or special character, you must use quotes.

`db.data.insertOne({'name' : 'anant', age : 29, course name : 'cs'})`

✗ Syntax Error: Unexpected identifier (because of space in 'course name')

Correct way: Use quotes around the field name if it contains space or special characters

`db.data.insertOne({'name' : 'anant', age : 29, 'course name' : 'cs'})`

Display all documents to verify insertion

`db.data.find()`

- MongoDB automatically creates an `_id` field
- `_id` is unique for every document
- Default type is `ObjectId`

----- ORDERED AND UNORDERED INSERT -----

➤ Ordered Insert (default)

```
db.users.insertMany(  
  [  
    { _id:1, name:"A"},  
    { _id:1, name:"B"}, // duplicate key  
    { _id:2, name:"C"}  
  ]  
)  
  
// Execution stops at error
```

➤ Unordered Insert :

```
db.users.insertMany(  
  [  
    { _id:1, name:"A"},  
    { _id:1, name:"B"},  
    { _id:2, name:"C"}  
  ],  
  { ordered: false }  
)  
  
// Error skipped, remaining documents inserted
```

----- FINDING DOCUMENTS -----

Find all documents where the field 'name' is equal to 'anant'

```
db.data.find({'name' : 'anant'})
```

Find only the first document that matches the condition (returns a single document)

```
db.data.findOne({'name' : 'anant'})
```

-Importing JSON in MongoDB:

1. Open the `shop` database → select the `products` collection.
2. Click on the "Add Data" button (top right corner).
3. Choose "Import File".
4. In the dialog that appears:
 - Click **Browse** and select your file, e.g.
`G:\MongoDB\MongoDB Import Files\products.json`
 - Choose the **File Type**:
 - If it's an array like `[ {...}, {...} ]`, select **JSON** and **JSON array**.
 - If it's CSV, select **CSV**.
5. Click **Import**.

----- COMPARISON OPERATORS -----

\$eq -> equal to

\$ne -> not equal to

\$gt -> greater than

\$gte -> greater than or equal to

\$lt -> less than

\$lte -> less than or equal to

\$in -> match any value from an array

\$nin -> exclude any value from an array

Find all products where price is exactly 699

```
shop> db.products.find({'price': {$eq: 699}})
```

Count how many products have price equal to 699

```
shop> db.products.find({'price': {$eq: 699}}).count()
```

Count how many products have price NOT equal to 699

```
shop> db.products.find({'price': {$ne: 699}}).count()
```

Count how many products have price greater than 699

```
shop> db.products.find({'price': {$gt: 699}}).count()
```

Count how many products have price greater than or equal to 699

```
shop> db.products.find({'price': {$gte: 699}}).count()
```


Count how many products have price less than 699

```
shop> db.products.find({'price': {$lt: 699}}).count()
```

Count how many products have price less than or equal to 699

```
shop> db.products.find({'price': {$lte: 699}}).count()
```

Find all products where price is 699, 129, or 69 (any of these values)

```
shop> db.products.find({'price': {$in: [699, 129, 69]}})
```

Find all products where price is NOT 699, 129, or 69

```
shop> db.products.find({'price': {$nin: [699, 129, 69]}})
```

----- CURSOR METHODS -----

Common cursor methods: count(), limit(), skip(), sort()

These methods are used to control and organize the result set.

Count how many products have price equal to 699

```
shop> db.products.find({'price': {$eq: 699}}).count()
```

Limit the output to only 1 document (useful for preview)

```
shop> db.products.find({'price': {$eq: 699}}).limit(1)
```

Skip the first document and then limit next 2 results

```
shop> db.products.find({'price': {$eq: 699}}).limit(2).skip(1)
```

Find products where price > 1250, limit to 3 results, and sort by price in ascending order

```
shop> db.products.find({'price': {$gt: 1250}}).limit(3).sort({'price': 1})
```

Find products where price > 1250, limit to 3 results, and sort by price in descending order

```
shop> db.products.find({'price': {$gt: 1250}}).limit(3).sort({'price': -1})
```

----- LOGICAL OPERATORS -----

\$and -> all conditions must be true

\$or -> at least one condition must be true

\$not -> negates (reverses) a condition

\$nor -> none of the conditions must be true

Find products where both conditions are true:

price > 0 AND name = 'Diamond Ring'

```
shop> db.products.find({$and: [{'price': {$gt: 0}}, {'name': 'Diamond Ring'}] })
```

Same as above using implicit \$and (MongoDB automatically applies AND for multiple conditions)

```
shop> db.products.find({'price': {$gt: 0}, 'name': 'Diamond Ring'})
```

Find products where price > 2000 OR name = 'Diamond Ring'

```
db.products.find({$or: [{'price': {$gt: 2000}}, {'name': 'Diamond Ring'}] })
```

Find products where neither price > 2000 nor name = 'Diamond Ring' (both conditions false)

```
db.products.find({$nor: [{'price': {$gt: 2000}}, {'name': 'Diamond Ring'}] })
```

Find and count products where price is NOT equal to 100 (using \$not)

```
shop> db.products.find({'price': {$not: {$eq: 100}}}).count()
```

OR (simpler version using \$ne - not equal)

```
shop> db.products.find({'price': {$ne: 100}}).count()
```

----- COMPLEX EXPRESSIONS (\$expr) -----

\$expr allows using aggregation expressions inside a `find()` query.

It lets you compare fields with other fields or perform calculations.

Find products where 'price' field is greater than 1340 (using \$expr)

```
shop> db.products.find({$expr: {$gt: ['$price', 1340]}})
```

OR simpler version (direct comparison without \$expr)

```
shop> db.products.find({'price': {$gt: 1340}})
```

Find sales where (quantity * price) is greater than targetPrice

```
shop> db.sales.find()
```

```
shop> db.sales.find({$expr: {$gt: [{multiply: ['$quantity', '$price']}, '$targetPrice']}})
```

Find sales where (quantity + price) is less than targetPrice

```
shop> db.sales.find({$expr: {$lt: [{add: ['$quantity', '$price']}, '$targetPrice']}})
```

----- ELEMENT OPERATORS -----

\$exists -> check if a field exists or not

\$type -> match documents based on field data type

\$size -> match array fields by their length (number of elements)

Find documents where the 'price' field exists

```
shop> db.products.find({price: {$exists: true}})
```

Find documents where the 'price' field does NOT exist

```
shop> db.products.find({price: {$exists: false}})
```

Count documents where 'price' field is of type number

```
shop> db.products.find({price: {$type: 'number'}}).count()
```

Count documents where 'price' field is of type string

```
shop> db.products.find({price: {$type: 'string'}}).count()
```

Count documents where 'isFeatured' field is of type boolean

```
shop> db.products.find({isFeatured: {$type: 'bool'}}).count()
```

Type 8 means "objectId" data type in MongoDB

```
shop> db.products.find({isFeatured: {$type: 8}}).count()
```

Find and count comments where array field 'comments' has exactly 2 elements

```
shop> db.comments.find({'comments': {'$size': 2}}).count()
```

Find and count comments where array field 'comments' has exactly 4 elements

```
shop> db.comments.find({'comments': {'$size': 4}}).count()
```

----- PROJECTION -----

Projection is used to control which fields are shown in the output.

1 = include field

0 = exclude field.

Find documents with comments array size = 2, show only comments field

```
shop> db.comments.find({'comments': {$size: 2}}, {comments: 1}).count()
```

Find documents with comments array size = 2, show only comments (hide _id)

```
shop> db.comments.find({'comments': {$size: 2}}, {comments: 1, _id: 0})
```

✗ Mixing inclusion and exclusion (except for _id) gives error

```
shop> db.comments.find({'comments': {$size: 2}}, {title: 1, author: 0, _id: 0})
```

Error: MongoServerError[Location31254]: Cannot do exclusion on field author in inclusion projection

Exclude title and author fields, show only _id

```
shop> db.comments.find({'comments': {$size: 2}}, {title: 0, author: 0, _id: 1})
```

Include title, author, and _id fields

```
shop> db.comments.find({'comments': {$size: 2}}, {title: 1, author: 1, _id: 1})
```

----- EMBEDDED DOCUMENTS -----

Used to query inside nested or embedded documents.

Find comments where user = 'Alice' (inside comments array)

```
shop> db.comments.find({'comments.user': 'Alice'})
```

Find comments where metadata.views > 1200

```
shop> db.comments.find({'metadata.views': {'$gt': 1200}})
```

Find comments where user = 'Henry' AND metadata.likes > 50

```
shop> db.comments.find({'comments.user': 'Henry', 'metadata.likes': {'$gt': 50}})
```

Find comments where user = 'Alice' AND metadata.likes > 50

```
shop> db.comments.find({'comments.user': 'Alice', 'metadata.likes': {'$gt': 50}})
```


----- \$ALL & \$ELEMATCH -----

\$all -> match if array contains all specified elements

\$elemMatch -> match single element that meets multiple conditions

Find comments where comments.user array has both 'Alice' and 'Vinod'

```
shop> db.comments.find({'comments.user': {$all: ['Alice', 'Vinod']}})
```

Find comments where a single array element matches both conditions

```
shop> db.comments.find({'comments': {$elemMatch: {'user': 'Vinod', 'text': 'Thanks for sharing.'}}})
```

OR simpler version using dot notation

```
shop> db.comments.find({'comments.user': 'Vinod', 'comments.text': 'Thanks for sharing.'})
```

----- UPDATE OPERATIONS -----

Common operations: updateOne(), updateMany(), insertOne(), insertMany()

\$set -> update specific field

\$inc -> increment value

\$rename -> rename field

\$unset -> remove field

\$push -> add to array

\$pop -> remove from array

Display all products

```
shop> db.products.find()
```

Find a specific product using its ObjectId

```
shop> db.products.find({_id: ObjectId('64c2363be32f4a51b19b9275')})
```

Update one document by ObjectId and change price to 450

```
shop> db.products.updateOne({_id: ObjectId("64c2363be32f4a51b19b9275")},  
{ $set: { 'price': 450 } })
```

Update one document where name = 'Designer Handbag' and set isFeatured = false

```
shop> db.products.find({name: 'Designer Handbag'})
```

```
shop> db.products.updateOne({name: 'Designer Handbag'}, { $set:  
{ 'isFeatured': false } })
```

Update all documents where price = 120 and set isFeatured = true

```
shop> db.products.updateMany({price: 120}, { $set: { 'isFeatured': true } })
```

----- REMOVE AND RENAME FIELDS -----

Count documents where price = 123

```
shop> db.products.find({'price': 123}).count()
```

Rename field 'isFeatured' to 'isFeature' in documents where price = 123

```
shop> db.products.updateMany({price: 123}, {$rename: {'isFeatured':  
'isFeature'}})
```

Remove a specific field using \$unset

```
db.collectionName.updateOne({filter}, {$unset: {fieldname: 1}})  
shop> db.comments.updateOne({_id: 1}, {$unset: {'user': 1}})
```

----- UPDATING ARRAYS AND EMBEDDED DOCUMENTS -----

Show one document from comments collection

```
shop> db.comments.find().limit(1)
```

Update field 'user' in first document

```
shop> db.comments.updateOne({_id: 1}, {$set: {'user': 'jessy'}})
```

Add a new element to comments array using \$push

```
shop> db.comments.updateOne({_id: 1}, {$push: {comments: {user: 'Eva', text: 'Subscribe the channel.'}}})
```

Display one document to confirm changes

```
shop> db.comments.find().limit(1)
```

Remove the last element from comments array using \$pop (1 = last element, -1 = first element)

```
shop> db.comments.updateOne({_id: 1}, {$pop: {'comments': 1}})
```

Update text of specific comment in array using positional operator (\$)

```
shop> db.comments.updateOne({_id: 7, 'comments.user': 'Alice'}, {$set: {'comments.$.text': 'Awesome Coding.'}})
```

Verify the update

```
shop> db.comments.find({_id: 7, 'comments.user': 'Alice'})
```

----- DELETE OPERATIONS -----

deleteOne() -> delete single document matching filter

deleteMany() -> delete all documents matching filter

```
shop> db.collectionName.deleteOne({filter})
```

```
shop> db.sales.deleteMany({price: 55})
```

----- INDEXES -----

Indexes improve query performance by allowing MongoDB to quickly locate data.

By default, MongoDB creates an index on the `_id` field.

View all existing indexes in the products collection

```
shop> db.products.getIndexes()
```

Create an ascending index on the 'name' field (1 = ascending, -1 = descending)

```
shop> db.products.createIndex({'name': 1})
```

View indexes after creating the new one

```
shop> db.products.getIndexes()
```

----- DROP INDEX -----

View current indexes

```
shop> db.products.getIndexes()
```

Drop (remove) index on 'name' field

```
shop> db.products.dropIndex({'name': 1})
```

View indexes again to confirm removal

```
shop> db.products.getIndexes()
```

----- AGGREGATION FRAMEWORK -----

Aggregation is used for data analysis and transformation.

Common stages: \$match, \$group, \$sort, \$project, \$unwind, \$addToSet, \$limit, \$skip, \$filter

Filter documents where name = 'Sleek Wooden Tuna'

```
shop> db.products.aggregate([{$match: {'name': 'Sleek Wooden Tuna'}}])
```

Filter documents where price > 1250

```
shop> db.products.aggregate([{$match: {price: {$gt: 1250}}})
```

Group products by company and count total number of products

```
shop> db.products.aggregate([{$group: {_id: "$company", totalProducts: {$sum: 1}}})
```

Match products from a specific company by ID

```
shop> db.products.aggregate([{$match: {company: '64c23350e32f4a51b19b9245'}}])
```

Group products by company and sum the total price

```
shop> db.products.aggregate([{$group: {_id: "$company", totalProducts: {$sum: "$price"}}})
```

Match products with price > 900, then group by company and calculate total price

```
shop> db.products.aggregate([  
  { $match: {price: {$gt: 900}} },  
  { $group: {_id: "$company", totalProducts: {$sum: "$price"}} }  
])
```

Match products with price > 1200, then group by company and calculate total price

```
shop> db.products.aggregate([  
  { $match: {price: {$gt: 1200}} },  
  { $group: {_id: "$company", totalProducts: {$sum: "$price"}} }  
])
```

Group sales data where quantity = 5, calculate total and average price

```
shop> db.sales.aggregate([  
  { $match: {quantity: 5} },  
  { $group: {_id: '$quantity', priceTotal: {$sum: '$price'}, priceAvg: {$avg:  
'$price'}} }  
])
```


Match products with price > 1200, group by category, sum price, and sort descending

```
shop> db.products.aggregate([  
  { $match: {price: {$gt: 1200}} },  
  { $group: {_id: "$category", totalPrice: {$sum: "$price"}} },  
  { $sort: {totalPrice: -1} }  
])
```

Match products with price < 1200, group by category, sum price, and sort descending

```
shop> db.products.aggregate([  
  { $match: {price: {$lt: 1200}} },  
  { $group: {_id: "$category", totalPrice: {$sum: "$price"}} },  
  { $sort: {totalPrice: -1} }  
])
```

----- \$LOOKUP (JOIN IN AGGREGATION) -----

\$lookup is used to join data from another collection.

Join orders collection with users collection

```
shop> db.orders.aggregate([  
  {  
    $lookup: {  
      from: "users",  
      localField: "userId",  
      foreignField: "_id",  
      as: "userDetails"  
    }  
  }  
])
```

----- \$PROJECT STAGE -----

Used to include/exclude fields and create computed fields in output.

Show only 'price' field

```
shop> db.products.aggregate([{$project: {price: 1}}])
```

Show only 'price' field and exclude _id

```
shop> db.products.aggregate([{$project: {price: 1, _id: 0}}])
```

Add new computed field 'discountPrice' (20% off)

```
shop> db.products.aggregate([{$project: {price: 1, discountPrice: {$multiply: ['$price', 0.8]}} }])
```

Match price > 1200, then project price and discountPrice

```
shop> db.products.aggregate([
  {$match: {price: {$gt: 1200}} },
  {$project: {price: 1, discountPrice: {$multiply: ['$price', 0.8]}} }
])
```

----- \$GROUP WITH ARRAYS -----

Group by price and push all 'colors' arrays into one array

```
shop> db.products.aggregate([  
  { $match: {price: {$gt: 1200}} },  
  { $group: {_id: '$price', allColors: {$push: '$colors'}} }  
])
```

Group by category and push all product names into one array

```
shop> db.products.aggregate([  
  { $match: {price: {$gt: 500}} },  
  { $group: {_id: '$category', allProducts: {$push: '$name'}} }  
])
```

----- \$UNWIND STAGE -----

\$unwind breaks array elements into separate documents.

Unwind colors array, then group by price and push all colors

```
shop> db.products.aggregate([  
  { $unwind: '$colors' },  
  { $match: {price: {$gt: 1200}} },  
  { $group: {_id: '$price', allColors: {$push: '$colors'}} }  
])
```

Same as above, but only unique colors using \$addToSet

```
shop> db.products.aggregate([  
  { $unwind: '$colors' },  
  { $match: {price: {$gt: 1200}} },  
  { $group: {_id: '$price', allColors: {$addToSet: '$colors'}} }  
])
```

Group by price, create unique color array, and show array length using \$size

```
db.products.aggregate([
  { $match: {price: {$gt: 1200}} },
  { $unwind: '$colors' },
  { $group: {_id: {priceGroup: '$price'}, colors: {$addToSet: '$colors'}} },
  { $project: {_id: 1, colors: 1, colorLength: {$size: '$colors'}} }
])
```

Add \$limit to show only first document

```
db.products.aggregate([
  { $match: {price: {$gt: 1200}} },
  { $unwind: '$colors' },
  { $group: {_id: {priceGroup: '$price'}, colors: {$addToSet: '$colors'}} },
  { $project: {_id: 1, colors: 1, colorLength: {$size: '$colors'}} },
  { $limit: 1 }
])
```

Add \$skip to skip first document

```
db.products.aggregate([
  { $match: {price: {$gt: 1200}} },
  { $unwind: '$colors' },
  { $group: {_id: {priceGroup: '$price'}, colors: {$addToSet: '$colors'}} },
  { $project: {_id: 1, colors: 1, colorLength: {$size: '$colors'}} },
  { $skip: 1 }
])
```

----- \$FILTER OPERATOR -----

Used to filter array elements based on a condition inside \$project.

```
shop> db.col.aggregate([
  { $project: {
    name: 1,
    thapaValue: {
      $filter: {
        input: '$values',   array field name
        as: 'val',          variable name for each element
        cond: { $gt: ['$val', 30] } condition to keep values > 30
      }
    }
  }
}]
```

----- \$FACET (MULTIPLE PIPELINES IN AGGREGATION) -----

\$facet allows running multiple aggregation pipelines in a single query.

Example: Get expensive products and cheap products together

```
shop> db.products.aggregate([
  {
    $facet: {
      expensiveProducts: [
        { $match: { price: { $gt: 1200 } } }
      ],
      cheapProducts: [
        { $match: { price: { $lt: 1200 } } }
      ]
    }
  }
])
```


----- TRANSACTIONS (ACID OPERATIONS) -----

Transactions are used to execute multiple operations atomically.

(All operations succeed or all fail)

Start a session

```
shop> s = db.getMongo().startSession()
```

Start transaction

```
shop> s.startTransaction()
```

Get collections from session

```
shop> products = s.getDatabase("shop").products
```

```
shop> sales = s.getDatabase("shop").sales
```

Update product price

```
shop> products.updateOne(  
  { name: "Designer Handbag" },  
  { $set: { price: 500 } }  
)
```

Insert related sales data

```
shop> sales.insertOne(  
  { product: "Designer Handbag", quantity: 1, price: 500 }  
)
```

Commit transaction

```
shop> s.commitTransaction()
```

End session

```
shop> s.endSession()
```

----- TTL INDEX (AUTOMATIC DATA EXPIRY) -----

TTL Index automatically deletes documents after a fixed time.

Example: Auto delete old sales data after 1 hour (3600 seconds)

```
shop> db.sales.createIndex(  
  { createdAt: 1 },  
  { expireAfterSeconds: 3600 }  
)
```